



PILLAR VOICE



PILLAR VOICE

SCHOOL ELECTION MANAGEMENT SYSTEM

**NAME: AKSHITH PRASAD
VORUMBUDI
CLASS: 12 SECTION: A
SCHOOL ROLL NO.: 5**



NPS INTERNATIONAL MYSORE ACADEMIC YEAR 2025-26



ENGLISH INVESTIGATORY PROJECT

NAME: AKSHITH PRASAD VORUMBUDI

CLASS: 12 SECTION: A

SCHOOL ROLL NO.: 05

BOARD ROLL NO.: 18621784



NPS INTERNATIONAL MYSORE

CERTIFICATE

This is to certify that
Akshith Prasad Vorumbudi
has successfully completed the investigatory project
in Computer Science as prescribed by the
Central Board of Secondary Education for the AISSCE course
in the Computer Lab of this school for the year 2025-26.

Internal Examiner

External Examiner

Principal's Signature

Date: _____

School Seal:

ACKNOWLEDGEMENT

I would like to take this opportunity to express my deep sense of gratitude to all those people without whom this project could never have been completed.

I would like to express my sincere gratitude to my Principal, Ms. Akhila Dinesh, and the school for providing me with the facilities required to do my project. I am also thankful for her constant encouragement and motivation.

I am highly indebted to my Computer Science teacher Mr. Maheswara for his valuable guidance, constant encouragement and moral support, which has promoted my efforts in all the stages of this project work.

Finally, words are not sufficient to express gratitude to my family members and friends who helped me in finalising this project within the limited time frame.

Name of the Student: Akshith Prasad Vorumbudi

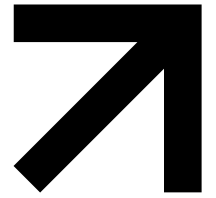
Class & Section: 12A



INVESTIGATORY
PROJECT

COMPUTER SCIENCE

Computer Science is the art of teaching logic to a machine, and in the process, learning the true logic of our own minds.



GRADE 12

COMPUTER SCIENCE

2025-26

CERTIFICATE



ACKNOWLEDGEMENT



MYSQL CODE OVERVIEW



PYTHON CODE OVERVIEW



BIBLIOGRAPHY





INTRODUCTION

Voting is more than just marking a choice, it is the voice of the people, a bridge between opinion and decision .

In schools, a voting system provides students the opportunity to choose their leaders who will represent their voice. This project aims to provide an organised and interactive platform to access detailed information about the School Election Management System. By leveraging Python and MySQL, this system enables users to analyse and manage voting data efficiently.

OBJECTIVE

- 1.To create a structured database of the voting system in schools.
- 2.To provide functionalities for retrieving, adding, updating, and deleting voting data.
- 3.To include details such as students, teachers, candidates, voters and their results.
- 4.To make the system user-friendly using Python-based programming for database interaction.
- 5.To aid students and teachers in decision-making by offering accurate and real-time data.





SCOPE

This project is designed for students, teachers and the school administration. It simplifies the process of accessing information about the school elections. The platform is scalable and can accommodate additional features, such as filtering students, teachers and admin based on user-defined criteria and generating analytical reports.

FEATURES

Search Functionality:

- Allow users to search for voting data such as students, teachers, candidates and votes from the school election.

User Interface:

- A user-friendly design ensures easy navigation and interaction with the system.

Reports and Analytics:

- Generate meaningful insights such as overall vote tallies, position-wise results, and house-wise analysis to support transparent decision-making.





HARDWARE REQUIREMENTS

PROCESSOR

Minimum: Dual-core processor (Intel i3 or equivalent).

Recommended: Quad-core processor or higher (Intel i5/i7, AMD Ryzen).

RAM

Minimum: 4 GB.

Recommended: 8 GB or more for better performance.

STORAGE

Minimum: 1 GB of free storage.

Recommended: SSD with 256 GB or more for faster file access.

DISPLAY

Minimum: 1366 x 768 resolution.

Recommended: Full HD (1920 x 1080) for better readability.

GRAPHICS

Integrated graphics are sufficient for this project.

PERIPHERALS - Keyboard and mouse. Internet connection for downloading software dependencies.





SOFTWARE REQUIREMENTS

OPERATING SYSTEM

Minimum: Linux(Ubuntu 20.04+)

Recommended: Windows 10/11,
macOS.

PYTHON

Minimum: Python 3.8

Recommended: Python 3.8 or later

REQUIRED LIBRARIES - pymysql or mysql-connector-python for
connecting to SQL

SQL DATABASE

Minimum: 1366 x 768 resolution.

Recommended: Full HD (1920 x
1080) for better readability.

MySQL (version 8.0 or later),
SQLite (built into Python)

PostgreSQL

IDE/Code Editor:

Visual Studio Code, PyCharm, or Jupyter Notebook for Python.

MySQL Workbench or pgAdmin for SQL queries and database
management.



MySQL CODE OVERVIEW



MySQL DATABASE:

```
mysql> show tables;
+-----+
| Tables_in_pillar_voice |
+-----+
| admins                  |
| candidates              |
| students                |
| teachers                |
| votes                   |
+-----+
5 rows in set (0.02 sec)
```

Shows a list of tables in the pillar_voice database:
admins,candidates,students,teachers,votes



```
mysql> desc admins;
+-----+-----+-----+-----+-----+-----+
| Field      | Type          | Null | Key | Default | Extra          |
+-----+-----+-----+-----+-----+-----+
| admin_id   | int           | NO   | PRI | NULL    | auto_increment |
| username   | varchar(50)   | YES  | UNI | NULL    |                |
| password   | varchar(50)   | YES  |     | NULL    |                |
+-----+-----+-----+-----+-----+-----+
3 rows in set (0.02 sec)
```

SQL query that shows the schema of the ADMINS table. The ADMINS table has columns for admin_id (an integer, primary key), username (varchar(50), unique), and password (varchar(50)).

```
mysql> desc students;
+-----+-----+-----+-----+-----+-----+
| Field      | Type          | Null | Key | Default | Extra          |
+-----+-----+-----+-----+-----+-----+
| student_id | int           | NO   | PRI | NULL    | auto_increment |
| name       | varchar(100)  | NO   |     | NULL    |                |
| class      | varchar(20)   | NO   |     | NULL    |                |
| username   | varchar(50)   | NO   | UNI | NULL    |                |
| password   | varchar(100)  | NO   |     | NULL    |                |
| house      | enum('Pioneers','Explorers','Challengers','Voyagers') | NO   |     | NULL    |                |
+-----+-----+-----+-----+-----+-----+
6 rows in set (0.01 sec)
```

SQL query that shows the schema of the STUDENTS table. The STUDENTS table has columns for student_id (an integer, primary key), name (varchar(100)), class (varchar(20)), username (varchar(50), unique), password (varchar(100)), and house (ENUM containing Pioneers, Explorers, Challengers, and Voyagers).





```
mysql> desc teachers;
+-----+-----+-----+-----+-----+-----+
| Field      | Type          | Null | Key | Default | Extra          |
+-----+-----+-----+-----+-----+-----+
| teacher_id | int           | NO   | PRI | NULL    | auto_increment |
| name       | varchar(100)  | NO   |     | NULL    |                |
| subject    | varchar(50)   | NO   |     | NULL    |                |
| username   | varchar(50)   | YES  | UNI | NULL    |                |
| password   | varchar(50)   | YES  |     | NULL    |                |
+-----+-----+-----+-----+-----+-----+
5 rows in set (0.00 sec)
```

SQL query that shows the schema of the TEACHERS table. The TEACHERS table has columns for teacher_id (an integer, primary key), name (varchar(100)), subject (varchar(50)), username (varchar(50), unique), and password (varchar(50)).

```
mysql> desc candidates;
+-----+-----+-----+-----+-----+-----+
| Field      | Type          | Null | Key | Default | Extra          |
+-----+-----+-----+-----+-----+-----+
| candidate_id | int           | NO   | PRI | NULL    | auto_increment |
| student_id  | int           | NO   | MUL | NULL    |                |
| name       | varchar(100)  | NO   |     | NULL    |                |
| position    | varchar(50)   | NO   |     | NULL    |                |
| house       | enum('Pioneers','Explorers','Challengers','Voyagers') | YES  |     | NULL    |                |
+-----+-----+-----+-----+-----+-----+
5 rows in set (0.01 sec)
```

SQL query that shows the schema of the CANDIDATES table. The CANDIDATES table has columns for candidate_id (an integer, primary key), student_id (an integer), name (varchar(100)), position (varchar(50)), and house (ENUM containing Pioneers, Explorers, Challengers, and Voyagers).



```
mysql> desc votes;
```

Field	Type	Null	Key	Default	Extra
vote_id	int	NO	PRI	NULL	auto_increment
student_id	int	NO	MUL	NULL	
candidate_id	int	NO	MUL	NULL	
position	varchar(50)	NO		NULL	
timestamp	datetime	YES		CURRENT_TIMESTAMP	DEFAULT_GENERATED

```
5 rows in set (0.05 sec)
```

SQL query that shows the schema of the VOTES table. The VOTES table has columns for vote_id (an integer, primary key), student_id (an integer), candidate_id (an integer), position (varchar(50)), and timestamp (datetime with default current timestamp).



ADMINS TABLE

```
mysql> select * from admins;
+-----+-----+-----+
| admin_id | username | password |
+-----+-----+-----+
|          1 | admin1    | pw_admin1 |
+-----+-----+-----+
1 row in set (0.00 sec)
```

SQL query that displays the admins table. It shows all administrator accounts with admin_id, username and password fields used to authenticate and manage the voting system.



STUDENTS TABLE

```
mysql> select * from students;
```

student_id	name	class	username	password	house
1	Aarav Gowda	6	stu6p1	pw_stu6p1	Pioneers
2	Diya Kapoor	6	stu6p2	pw_stu6p2	Pioneers
3	Kabir Nair	6	stu6p3	pw_stu6p3	Pioneers
4	Meera Joshi	6	stu6e1	pw_stu6e1	Explorers
5	Rohan Gupta	6	stu6e2	pw_stu6e2	Explorers
6	Ananya Iyer	6	stu6e3	pw_stu6e3	Explorers
7	Ishita Verma	6	stu6c1	pw_stu6c1	Challengers
8	Arjun Rao	6	stu6c2	pw_stu6c2	Challengers
9	Sneha Pillai	6	stu6c3	pw_stu6c3	Challengers
10	Rahul Sharma	6	stu6v1	pw_stu6v1	Voyagers
11	Priya Menon	6	stu6v2	pw_stu6v2	Voyagers
12	Dev Anand	6	stu6v3	pw_stu6v3	Voyagers
13	Sanya Reddy	7	stu7p1	pw_stu7p1	Pioneers
14	Vikram Das	7	stu7p2	pw_stu7p2	Pioneers
15	Nisha Bhat	7	stu7p3	pw_stu7p3	Pioneers
16	Aditya Jain	7	stu7e1	pw_stu7e1	Explorers
17	Kavya Singh	7	stu7e2	pw_stu7e2	Explorers
18	Manav Kulkarni	7	stu7e3	pw_stu7e3	Explorers
19	Tanvi Mishra	7	stu7c1	pw_stu7c1	Challengers
20	Aryan Patel	7	stu7c2	pw_stu7c2	Challengers
21	Ritika Sen	7	stu7c3	pw_stu7c3	Challengers
22	Neeraj Kumar	7	stu7v1	pw_stu7v1	Voyagers
23	Shreya Ghosh	7	stu7v2	pw_stu7v2	Voyagers
24	Parth Malhotra	7	stu7v3	pw_stu7v3	Voyagers
25	Rhea Thomas	8	stu8p1	pw_stu8p1	Pioneers
26	Karan Sethi	8	stu8p2	pw_stu8p2	Pioneers
27	Mitali Roy	8	stu8p3	pw_stu8p3	Pioneers
28	Siddharth Menon	8	stu8e1	pw_stu8e1	Explorers
29	Pooja Desai	8	stu8e2	pw_stu8e2	Explorers
30	Varun Chatterjee	8	stu8e3	pw_stu8e3	Explorers



31	Aditi Sharma	8	stu8c1	pw_stu8c1	Challengers
32	Raghav Prasad	8	stu8c2	pw_stu8c2	Challengers
33	Snehal Patil	8	stu8c3	pw_stu8c3	Challengers
34	Nikhil Arora	8	stu8v1	pw_stu8v1	Voyagers
35	Simran Kaur	8	stu8v2	pw_stu8v2	Voyagers
36	Harsh Vyas	8	stu8v3	pw_stu8v3	Voyagers
37	Aniket Sharma	9	stu9p1	pw_stu9p1	Pioneers
38	Riya Nair	9	stu9p2	pw_stu9p2	Pioneers
39	Sahil Mehta	9	stu9p3	pw_stu9p3	Pioneers
40	Ishaan Kapoor	9	stu9e1	pw_stu9e1	Explorers
41	Tanya Desai	9	stu9e2	pw_stu9e2	Explorers
42	Kunal Joshi	9	stu9e3	pw_stu9e3	Explorers
43	Sneha Reddy	9	stu9c1	pw_stu9c1	Challengers
44	Raghav Menon	9	stu9c2	pw_stu9c2	Challengers
45	Maya Bhat	9	stu9c3	pw_stu9c3	Challengers
46	Arnav Gupta	9	stu9v1	pw_stu9v1	Voyagers
47	Shruti Iyer	9	stu9v2	pw_stu9v2	Voyagers
48	Devansh Rao	9	stu9v3	pw_stu9v3	Voyagers
49	Krishna Patel	10	stu10p1	pw_stu10p1	Pioneers
50	Aditi Sen	10	stu10p2	pw_stu10p2	Pioneers
51	Rohan Malhotra	10	stu10p3	pw_stu10p3	Pioneers
52	Nikita Das	10	stu10e1	pw_stu10e1	Explorers
53	Yash Arora	10	stu10e2	pw_stu10e2	Explorers
54	Pallavi Mehta	10	stu10e3	pw_stu10e3	Explorers
55	Rajat Kapoor	10	stu10c1	pw_stu10c1	Challengers
56	Sanya Kulkarni	10	stu10c2	pw_stu10c2	Challengers
57	Vivek Reddy	10	stu10c3	pw_stu10c3	Challengers
58	Anushka Nair	10	stu10v1	pw_stu10v1	Voyagers
59	Pranav Joshi	10	stu10v2	pw_stu10v2	Voyagers
60	Divya Sharma	10	stu10v3	pw_stu10v3	Voyagers
61	Aarushi Menon	11	stu11p1	pw_stu11p1	Pioneers
62	Raghav Sethi	11	stu11p2	pw_stu11p2	Pioneers
63	Snehal Desai	11	stu11p3	pw_stu11p3	Pioneers
64	Karan Bhat	11	stu11e1	pw_stu11e1	Explorers
65	Ishita Rao	11	stu11e2	pw_stu11e2	Explorers
66	Manish Gupta	11	stu11e3	pw_stu11e3	Explorers



67	Tanvi Iyer	11	stu11c1	pw_stu11c1	Challengers
68	Aryan Malhotra	11	stu11c2	pw_stu11c2	Challengers
69	Ritika Sen	11	stu11c3	pw_stu11c3	Challengers
70	Neha Kapoor	11	stu11v1	pw_stu11v1	Voyagers
71	Aditya Das	11	stu11v2	pw_stu11v2	Voyagers
72	Simran Reddy	11	stu11v3	pw_stu11v3	Voyagers
73	Akshith Rao	12	stu12p1	pw_stu12p1	Pioneers
74	Diya Menon	12	stu12p2	pw_stu12p2	Pioneers
75	Kabir Sharma	12	stu12p3	pw_stu12p3	Pioneers
76	Meera Patel	12	stu12e1	pw_stu12e1	Explorers
77	Rohan Nair	12	stu12e2	pw_stu12e2	Explorers
78	Ananya Gupta	12	stu12e3	pw_stu12e3	Explorers
79	Ishita Sen	12	stu12c1	pw_stu12c1	Challengers
80	Arjun Malhotra	12	stu12c2	pw_stu12c2	Challengers
81	Sneha Das	12	stu12c3	pw_stu12c3	Challengers
82	Rahul Mehta	12	stu12v1	pw_stu12v1	Voyagers
83	Priya Iyer	12	stu12v2	pw_stu12v2	Voyagers
84	Dev Arora	12	stu12v3	pw_stu12v3	Voyagers

84 rows in set (0.01 sec)

SQL query that displays the students table. It lists every registered student with student_id, name, class, login credentials (username, password) and house so the system can identify voters and enforce house-based rules.

TEACHERS TABLE

```
mysql> select * from teachers;
```

teacher_id	name	subject	username	password
1	Mr. Rajesh Rao	Computer Science	teach1	pw_teach1
2	Ms. Meera Nair	Mathematics	teach2	pw_teach2
3	Mr. Anil Sharma	Physics	teach3	pw_teach3
4	Ms. Kavita Iyer	Chemistry	teach4	pw_teach4
5	Mr. Suresh Menon	Biology	teach5	pw_teach5
6	Ms. Priya Kapoor	English	teach6	pw_teach6
7	Mr. Vivek Das	History	teach7	pw_teach7
8	Ms. Shalini Verma	Geography	teach8	pw_teach8
9	Mr. Arvind Gupta	Economics	teach9	pw_teach9
10	Ms. Ritu Sen	Political Science	teach10	pw_teach10
11	Mr. Manoj Bhat	Physical Education	teach11	pw_teach11
12	Ms. Neha Malhotra	Art & Culture	teach12	pw_teach12
13	Mr. Sanjay Kulkarni	Hindi	teach13	pw_teach13
14	Ms. Rekha Choudhary	Sanskrit	teach14	pw_teach14
15	Mr. Ajay Prasad	Commerce	teach15	pw_teach15
16	Ms. Bhavna Reddy	Business Studies	teach16	pw_teach16
17	Mr. Deepak Mishra	Psychology	teach17	pw_teach17
18	Ms. Anjali Sinha	Sociology	teach18	pw_teach18
19	Mr. Harish Venkatesh	Music	teach19	pw_teach19

19 rows in set (0.00 sec)

SQL query that displays the teachers table. It returns all teacher records with teacher_id, name, subject, and login credentials (username, password) for staff access and oversight.

CANDIDATES TABLE

```
mysql> select * from candidates;
```

candidate_id	student_id	name	position	house
1	49	Krishna Patel	House Captain	Pioneers
2	50	Aditi Sen	House Captain	Pioneers
3	52	Nikita Das	House Captain	Explorers
4	53	Yash Arora	House Captain	Explorers
5	55	Rajat Kapoor	House Captain	Challengers
6	56	Sanya Kulkarni	House Captain	Challengers
7	58	Anushka Nair	House Captain	Voyagers
8	59	Pranav Joshi	House Captain	Voyagers
9	61	Raghav Sethi	Sports Captain	Pioneers
10	64	Ishita Rao	Sports Captain	Explorers
11	67	Tanvi Iyer	Sports Captain	Challengers
12	70	Neha Kapoor	Sports Captain	Voyagers
13	73	Akshith Rao	School Prefect	Pioneers
14	76	Meera Patel	School Prefect	Explorers
15	79	Ishita Sen	School Prefect	Challengers
16	82	Rahul Mehta	School Prefect	Voyagers

16 rows in set (0.00 sec)

SQL query that displays the candidates table. It shows all election candidates with candidate_id, linked student_id, candidate name, the position they are contesting (e.g., House Captain, Sports Captain, School Prefect) and their house.



TABLES SUMMARY

students	Stores the details of all registered students, including their class, login credentials, and house allocation.
teachers	Contains information about teachers, their subjects, and their login credentials for system access.
admin	Holds the administrator account details used to manage and oversee the entire voting system.
candidates	Lists all student candidates standing for various positions along with their house and student ID.
votes	Records each vote cast, linking students to their chosen candidates along with a timestamp.



PYTHON CODE OVERVIEW



```
import mysql.connector

# ----- DB CONNECTION -----
def get_connection():
    return mysql.connector.connect(
        host="localhost",
        user="root",
        password="root",
        database="pillar_voice"
    )
```

```
# ----- LOGIN -----
def student_login(u, p):
    conn = get_connection(); cur = conn.cursor()
    cur.execute("SELECT student_id, house FROM students WHERE username=%s AND password=%s", (u, p))
    r = cur.fetchone(); conn.close()
    return {"student_id": r[0], "house": r[1]} if r else None

def teacher_login(u, p):
    conn = get_connection(); cur = conn.cursor()
    cur.execute("SELECT teacher_id FROM teachers WHERE username=%s AND password=%s", (u, p))
    r = cur.fetchone(); conn.close()
    return {"teacher_id": r[0]} if r else None

def admin_login(u, p):
    conn = get_connection(); cur = conn.cursor()
    cur.execute("SELECT admin_id FROM admins WHERE username=%s AND password=%s", (u, p))
    r = cur.fetchone(); conn.close()
    return {"admin_id": r[0]} if r else None
```

1. Connecting MySQL with Python
2. Defining Function - **student_login()** for authentication of a student
3. Defining Function - **teacher_login()** for authentication of a teacher
4. Defining Function - **admin_login()** for authentication of an admin

```

# ----- MENUS -----
def student_menu(s):
    while True:
        print("\n--- Student Menu ---")
        print("1. House Captain\n2. Sports Captain\n3. School Prefect\n4. Exit")
        ch = input("Choice: ").strip()
        if ch == "1": cast_vote(s["student_id"], "student", s["house"], "House Captain")
        elif ch == "2": cast_vote(s["student_id"], "student", s["house"], "Sports Captain")
        elif ch == "3": cast_vote(s["student_id"], "student", s["house"], "School Prefect")
        elif ch == "4": break

def teacher_menu(t):
    while True:
        print("\n--- Teacher Menu ---")
        print("1. Sports Captain\n2. School Prefect\n3. Exit")
        ch = input("Choice: ").strip()
        if ch == "1": cast_vote(t["teacher_id"], "teacher", None, "Sports Captain")
        elif ch == "2": cast_vote(t["teacher_id"], "teacher", None, "School Prefect")
        elif ch == "3": break

def admin_menu(a):
    while True:
        print("\n--- Admin Menu ---")
        print("1. Overall tally")
        print("2. House Captain tallies (by house)")
        print("3. Sports Captain tally")
        print("4. School Prefect tally")
        print("5. View Students")
        print("6. View Teachers")
        print("7. View Candidates")
        print("8. Alter entry")
        print("9. Delete entry")
        print("10. Exit")
        ch = input("Choice: ").strip()
        if ch == "1": show_overall_tally()
        elif ch == "2":
            for h in ["Pioneers", "Explorers", "Challengers", "Voyagers"]:
                show_house_tally(h)
        elif ch == "3": show_position_tally("Sports Captain")
        elif ch == "4": show_position_tally("School Prefect")
        elif ch == "5": show_table("students", ["student_id", "name", "house", "username"])
        elif ch == "6": show_table("teachers", ["teacher_id", "name", "subject", "username"])
        elif ch == "7": show_table("candidates", ["candidate_id", "name", "position", "house"])
        elif ch == "8": alter_entry()
        elif ch == "9": delete_entry()
        elif ch == "10": break

```

5. Defining Function - **student_menu()** for displaying a menu for students
6. Defining Function - **teacher_menu()** for displaying a menu for teachers
7. Defining Function - **admin_menu()** for displaying a menu for admin

```

# ----- VOTING -----
def cast_vote(voter_id, voter_type, house, position):
    conn = get_connection(); cur = conn.cursor()

    # Candidate list per role/position
    if position == "House Captain":
        if voter_type == "teacher":
            print("X Teachers cannot vote for House Captain."); conn.close(); return
        cur.execute("SELECT candidate_id, name FROM candidates WHERE position=%s AND house=%s",
                    (position, house))
    else:
        cur.execute("SELECT candidate_id, name FROM candidates WHERE position=%s", (position,))

    candidates = cur.fetchall()
    if not candidates:
        print("Δ No candidates found."); conn.close(); return

    print(f"\nCandidates for {position}:")
    for c in candidates: print(f"{c[0]}. {c[1]}")

    try:
        cid = int(input("Enter candidate_id to vote: ").strip())
        if cid not in [c[0] for c in candidates]:
            print("X Invalid choice."); conn.close(); return
        cur.execute("INSERT INTO votes (student_id, candidate_id, position) VALUES (%s, %s, %s)",
                    (voter_id, cid, position))
        conn.commit(); print("✓ Vote recorded.")
    except Exception as e:
        print("X Error:", e)
    finally:
        conn.close()

```

8. Defining Function - **caste_vote()** for casting vote in the school election

```
# ----- ADMIN VIEWS -----
def show_overall_tally():
    conn = get_connection(); cur = conn.cursor()
    cur.execute("""SELECT c.position, c.name, c.house, COUNT(v.vote_id) AS votes
                  FROM candidates c
                  LEFT JOIN votes v ON c.candidate_id = v.candidate_id
                  GROUP BY c.position, c.name, c.house
                  ORDER BY c.position, votes DESC, c.name""")
    print("\nOverall Tally:")
    for r in cur.fetchall(): print(r)
    conn.close()

def show_house_tally(house):
    conn = get_connection(); cur = conn.cursor()
    cur.execute("""SELECT c.name, COUNT(v.vote_id) AS votes
                  FROM candidates c
                  LEFT JOIN votes v ON c.candidate_id = v.candidate_id
                  WHERE c.position='House Captain' AND c.house=%s
                  GROUP BY c.name
                  ORDER BY votes DESC, c.name""", (house,))
    print(f"\n{house} House Captain Tally:")
    for r in cur.fetchall(): print(r)
    conn.close()

def show_position_tally(position):
    conn = get_connection(); cur = conn.cursor()
    cur.execute("""SELECT c.name, c.house, COUNT(v.vote_id) AS votes
                  FROM candidates c
                  LEFT JOIN votes v ON c.candidate_id = v.candidate_id
                  WHERE c.position=%s
                  GROUP BY c.name, c.house
                  ORDER BY votes DESC, c.name""", (position,))
    print(f"\n{position} Tally:")
    for r in cur.fetchall(): print(r)
    conn.close()

def show_table(table, columns):
    conn = get_connection(); cur = conn.cursor()
    cur.execute(f"SELECT {', '.join(columns)} FROM {table} ORDER BY {columns[0]}")
    print(f"\n{table.capitalize()} Table:")
    for r in cur.fetchall(): print(r)
    conn.close()
```

9. Defining Function - **show_overall_tally()** for displaying the result of all the school elections
10. Defining Function - **show_house_tally()** for displaying the result of the house elections
11. Defining Function - **show_position_tally()** for displaying the result for a specific position
12. Defining Function - **show_table()** for displaying selected columns from any table in an ordered format

```
# ----- ALTER / DELETE -----
def alter_entry():
    table = input("Table (students/teachers/candidates): ").strip()
    id_field = "student_id" if table=="students" else "teacher_id" if table=="teachers" else "candidate_id"
    rid = input(f"{id_field}: ").strip()
    field = input("Field to update: ").strip()
    val = input("New value: ").strip()
    conn = get_connection(); cur = conn.cursor()
    try:
        cur.execute(f"UPDATE {table} SET {field}=%s WHERE {id_field}=%s", (val, rid))
        conn.commit(); print("✅ Updated.")
    except Exception as e:
        print("❌ Error:", e)
    finally:
        conn.close()

def delete_entry():
    table = input("Table (students/teachers/candidates): ").strip()
    id_field = "student_id" if table=="students" else "teacher_id" if table=="teachers" else "candidate_id"
    rid = input(f"{id_field}: ").strip()
    conn = get_connection(); cur = conn.cursor()
    try:
        cur.execute(f"DELETE FROM {table} WHERE {id_field}=%s", (rid,))
        conn.commit(); print("✅ Deleted.")
    except Exception as e:
        print("❌ Error:", e)
    finally:
        conn.close()
```

```
# ----- DEDUPE CANDIDATES -----
def dedupe_candidates():
    """
    Keeps the earliest candidate_id per (name, position, house).
    Removes exact duplicates that cause repeated rows in tallies.
    """
    conn = get_connection(); cur = conn.cursor()
    try:
        cur.execute("""
            DELETE c1 FROM candidates c1
            INNER JOIN candidates c2
            ON c1.name = c2.name
            AND c1.position = c2.position
            AND c1.house = c2.house
            AND c1.candidate_id > c2.candidate_id;
        """)
        conn.commit(); print("✅ Duplicate candidates removed.")
    except Exception as e:
        print("❌ Error:", e)
    finally:
        conn.close()
```

13. Defining Function - **alter_entry()** for updating a record.
14. Defining Function - **delete_entry()** for deleting a record
15. Defining Function - **dedupe_candidates()** for removing duplicate candidate records

```

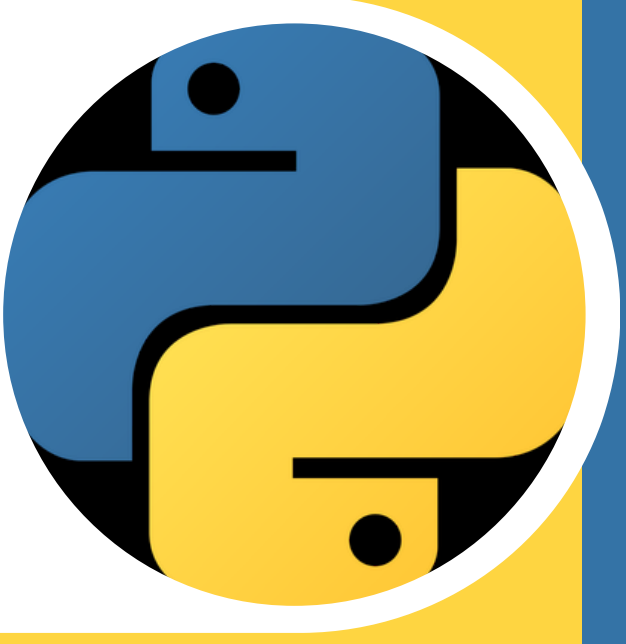
# ----- MAIN -----
if __name__ == "__main__":
    print("=== School Voting System ===")
    u = input("Username: ").strip()
    p = input("Password: ").strip()

    s = student_login(u, p)
    if s:
        print("✅ Student login"); student_menu(s)
    else:
        t = teacher_login(u, p)
        if t:
            print("✅ Teacher login"); teacher_menu(t)
        else:
            a = admin_login(u, p)
            if a:
                print("✅ Admin login"); admin_menu(a)
            else:
                print("❌ Invalid username or password.")

```

16. The main section controls program flow. It authenticates the user as a student, teacher, or admin and redirects them to the respective menu. Invalid credentials are rejected securely.

OUTPUT



```
=== School Voting System ===
Username: stu12v3
Password: pw_stu12v3
☒ Student login

--- Student Menu ---
1. House Captain
2. Sports Captain
3. School Prefect
4. Exit
Choice: 1

Candidates for House Captain:
7. Anushka Nair
8. Pranav Joshi
Enter candidate_id to vote: 7
☒ Vote recorded.

--- Student Menu ---
1. House Captain
2. Sports Captain
3. School Prefect
4. Exit
Choice: 2

Candidates for Sports Captain:
9. Raghav Sethi
10. Ishita Rao
11. Tanvi Iyer
12. Neha Kapoor
Enter candidate_id to vote: 9
☒ Vote recorded.

--- Student Menu ---
1. House Captain
2. Sports Captain
3. School Prefect
4. Exit
Choice: 3

Candidates for School Prefect:
13. Rahul Rao
14. Meera Patel
15. Ishita Sen
16. Rahul Mehta
Enter candidate_id to vote: 15
☒ Vote recorded.

--- Student Menu ---
1. House Captain
2. Sports Captain
3. School Prefect
4. Exit
Choice: 4
```

=== School Voting System ===

Username: teach1

Password: pw_teach1

☒ Teacher login

--- Teacher Menu ---

1. Sports Captain

2. School Prefect

3. Exit

Choice: 1

Candidates for Sports Captain:

9. Raghav Sethi

10. Ishita Rao

11. Tanvi Iyer

12. Neha Kapoor

Enter candidate_id to vote: 10

☒ Vote recorded.

--- Teacher Menu ---

1. Sports Captain

2. School Prefect

3. Exit

Choice: 2

Candidates for School Prefect:

13. Rahul Rao

14. Meera Patel

15. Ishita Sen

16. Rahul Mehta

Enter candidate_id to vote: 15

☒ Vote recorded.

--- Teacher Menu ---

1. Sports Captain

2. School Prefect

3. Exit

Choice: 3

=== School Voting System ===

Username: admin1

Password: pw_admin1

☒ Admin login

--- Admin Menu ---

1. Overall tally
 2. House Captain tallies (by house)
 3. Sports Captain tally
 4. School Prefect tally
 5. View Students
 6. View Teachers
 7. View Candidates
 8. Alter entry
 9. Delete entry
 10. Exit
- Choice: 1

Overall Tally:

('House Captain', 'Anushka Nair', 'Voyagers', 1)
('House Captain', 'Aditi Sen', 'Pioneers', 0)
('House Captain', 'Nikita Das', 'Explorers', 0)
('House Captain', 'Patel Krishna', 'Pioneers', 0)
('House Captain', 'Pranav Joshi', 'Voyagers', 0)
('House Captain', 'Rajat Kapoor', 'Challengers', 0)
('House Captain', 'Sanya Kulkarni', 'Challengers', 0)
('House Captain', 'Yash Arora', 'Explorers', 0)
('School Prefect', 'Ishita Sen', 'Challengers', 2)
('School Prefect', 'Meera Patel', 'Explorers', 0)
('School Prefect', 'Rahul Mehta', 'Voyagers', 0)
('School Prefect', 'Rahul Rao', 'Pioneers', 0)
('Sports Captain', 'Ishita Rao', 'Explorers', 1)
('Sports Captain', 'Raghav Sethi', 'Pioneers', 1)
('Sports Captain', 'Neha Kapoor', 'Voyagers', 0)
('Sports Captain', 'Tanvi Iyer', 'Challengers', 0)

--- Admin Menu ---

1. Overall tally
2. House Captain tallies (by house)
3. Sports Captain tally
4. School Prefect tally
5. View Students
6. View Teachers
7. View Candidates
8. Alter entry
9. Delete entry
10. Exit

Choice: 2

Pioneers House Captain Tally:

('Aditi Sen', 0)

('Patel Krishna', 0)

Explorers House Captain Tally:

('Nikita Das', 0)

('Yash Arora', 0)

Challengers House Captain Tally:

('Rajat Kapoor', 0)

('Sanya Kulkarni', 0)

Voyagers House Captain Tally:

('Anushka Nair', 1)

('Pranav Joshi', 0)

```
--- Admin Menu ---
1. Overall tally
2. House Captain tallies (by house)
3. Sports Captain tally
4. School Prefect tally
5. View Students
6. View Teachers
7. View Candidates
8. Alter entry
9. Delete entry
10. Exit
Choice: 3
```

```
Sports Captain Tally:
('Ishita Rao', 'Explorers', 1)
('Raghav Sethi', 'Pioneers', 1)
('Neha Kapoor', 'Voyagers', 0)
('Tanvi Iyer', 'Challengers', 0)
```

```
--- Admin Menu ---
1. Overall tally
2. House Captain tallies (by house)
3. Sports Captain tally
4. School Prefect tally
5. View Students
6. View Teachers
7. View Candidates
8. Alter entry
9. Delete entry
10. Exit
Choice: 4
```

```
School Prefect Tally:
('Ishita Sen', 'Challengers', 2)
('Meera Patel', 'Explorers', 0)
('Rahul Mehta', 'Voyagers', 0)
('Rahul Rao', 'Pioneers', 0)
```

--- Admin Menu ---

1. Overall tally
 2. House Captain tallies (by house)
 3. Sports Captain tally
 4. School Prefect tally
 5. View Students
 6. View Teachers
 7. View Candidates
 8. Alter entry
 9. Delete entry
 10. Exit
- Choice: 5

Students Table:

- | | | | |
|------|---------------------|----------------|-----------|
| (1, | 'Aarav Gowda', | 'Pioneers', | 'stu6p1') |
| (2, | 'Diya Kapoor', | 'Pioneers', | 'stu6p2') |
| (3, | 'Kabir Nair', | 'Pioneers', | 'stu6p3') |
| (4, | 'Meera Joshi', | 'Explorers', | 'stu6e1') |
| (5, | 'Rohan Gupta', | 'Explorers', | 'stu6e2') |
| (6, | 'Ananya Iyer', | 'Explorers', | 'stu6e3') |
| (7, | 'Ishita Verma', | 'Challengers', | 'stu6c1') |
| (8, | 'Arjun Rao', | 'Challengers', | 'stu6c2') |
| (9, | 'Sneha Pillai', | 'Challengers', | 'stu6c3') |
| (10, | 'Rahul Sharma', | 'Voyagers', | 'stu6v1') |
| (11, | 'Priya Menon', | 'Voyagers', | 'stu6v2') |
| (12, | 'Dev Anand', | 'Voyagers', | 'stu6v3') |
| (13, | 'Sanya Reddy', | 'Pioneers', | 'stu7p1') |
| (14, | 'Vikram Das', | 'Pioneers', | 'stu7p2') |
| (15, | 'Nisha Bhat', | 'Pioneers', | 'stu7p3') |
| (16, | 'Aditya Jain', | 'Explorers', | 'stu7e1') |
| (17, | 'Kavya Singh', | 'Explorers', | 'stu7e2') |
| (18, | 'Manav Kulkarni', | 'Explorers', | 'stu7e3') |
| (19, | 'Tanvi Mishra', | 'Challengers', | 'stu7c1') |
| (20, | 'Aryan Patel', | 'Challengers', | 'stu7c2') |
| (21, | 'Ritika Sen', | 'Challengers', | 'stu7c3') |
| (22, | 'Neeraj Kumar', | 'Voyagers', | 'stu7v1') |
| (23, | 'Shreya Ghosh', | 'Voyagers', | 'stu7v2') |
| (24, | 'Parth Malhotra', | 'Voyagers', | 'stu7v3') |
| (25, | 'Rhea Thomas', | 'Pioneers', | 'stu8p1') |
| (26, | 'Karan Sethi', | 'Pioneers', | 'stu8p2') |
| (27, | 'Mitali Roy', | 'Pioneers', | 'stu8p3') |
| (28, | 'Siddharth Menon', | 'Explorers', | 'stu8e1') |
| (29, | 'Pooja Desai', | 'Explorers', | 'stu8e2') |
| (30, | 'Varun Chatterjee', | 'Explorers', | 'stu8e3') |
| (31, | 'Aditi Sharma', | 'Challengers', | 'stu8c1') |
| (32, | 'Raghav Prasad', | 'Challengers', | 'stu8c2') |

(33, 'Snehal Patil', 'Challengers', 'stu8c3')
(34, 'Nikhil Arora', 'Voyagers', 'stu8v1')
(35, 'Simran Kaur', 'Voyagers', 'stu8v2')
(36, 'Harsh Vyas', 'Voyagers', 'stu8v3')
(37, 'Aniket Sharma', 'Pioneers', 'stu9p1')
(38, 'Riya Nair', 'Pioneers', 'stu9p2')
(39, 'Sahil Mehta', 'Pioneers', 'stu9p3')
(40, 'Ishaan Kapoor', 'Explorers', 'stu9e1')
(41, 'Tanya Desai', 'Explorers', 'stu9e2')
(42, 'Kunal Joshi', 'Explorers', 'stu9e3')
(43, 'Sneha Reddy', 'Challengers', 'stu9c1')
(44, 'Raghav Menon', 'Challengers', 'stu9c2')
(45, 'Maya Bhat', 'Challengers', 'stu9c3')
(46, 'Arnav Gupta', 'Voyagers', 'stu9v1')
(47, 'Shruti Iyer', 'Voyagers', 'stu9v2')
(48, 'Devansh Rao', 'Voyagers', 'stu9v3')
(49, 'Krishna Patel', 'Pioneers', 'stu10p1')
(50, 'Aditi Sen', 'Pioneers', 'stu10p2')
(51, 'Rohan Malhotra', 'Pioneers', 'stu10p3')
(52, 'Nikita Das', 'Explorers', 'stu10e1')
(53, 'Yash Arora', 'Explorers', 'stu10e2')
(54, 'Pallavi Mehta', 'Explorers', 'stu10e3')
(55, 'Rajat Kapoor', 'Challengers', 'stu10c1')
(56, 'Sanya Kulkarni', 'Challengers', 'stu10c2')
(57, 'Vivek Reddy', 'Challengers', 'stu10c3')
(58, 'Anushka Nair', 'Voyagers', 'stu10v1')
(59, 'Pranav Joshi', 'Voyagers', 'stu10v2')
(60, 'Divya Sharma', 'Voyagers', 'stu10v3')
(61, 'Aarushi Menon', 'Pioneers', 'stu11p1')
(62, 'Raghav Sethi', 'Pioneers', 'stu11p2')
(63, 'Snehal Desai', 'Pioneers', 'stu11p3')
(64, 'Karan Bhat', 'Explorers', 'stu11e1')
(65, 'Ishita Rao', 'Explorers', 'stu11e2')
(66, 'Manish Gupta', 'Explorers', 'stu11e3')
(67, 'Tanvi Iyer', 'Challengers', 'stu11c1')
(68, 'Aryan Malhotra', 'Challengers', 'stu11c2')
(69, 'Ritika Sen', 'Challengers', 'stu11c3')
(70, 'Neha Kapoor', 'Voyagers', 'stu11v1')
(71, 'Aditya Das', 'Voyagers', 'stu11v2')
(72, 'Simran Reddy', 'Voyagers', 'stu11v3')
(73, 'Akshith Rao', 'Pioneers', 'stu12p1')
(74, 'Diya Menon', 'Pioneers', 'stu12p2')
(75, 'Kabir Sharma', 'Pioneers', 'stu12p3')
(76, 'Meera Patel', 'Explorers', 'stu12e1')
(77, 'Rohan Nair', 'Explorers', 'stu12e2')

```
(78, 'Ananya Gupta', 'Explorers', 'stu12e3')
(79, 'Ishita Sen', 'Challengers', 'stu12c1')
(80, 'Arjun Malhotra', 'Challengers', 'stu12c2')
(81, 'Sneha Das', 'Challengers', 'stu12c3')
(82, 'Rahul Mehta', 'Voyagers', 'stu12v1')
(83, 'Priya Iyer', 'Voyagers', 'stu12v2')
(84, 'Dev Arora', 'Voyagers', 'stu12v3')
```

--- Admin Menu ---

1. Overall tally
 2. House Captain tallies (by house)
 3. Sports Captain tally
 4. School Prefect tally
 5. View Students
 6. View Teachers
 7. View Candidates
 8. Alter entry
 9. Delete entry
 10. Exit
- Choice: 6

Teachers Table:

```
(1, 'Mr. Rajesh Rao', 'Computer Science', 'teach1')
(2, 'Ms. Meera Nair', 'Mathematics', 'teach2')
(3, 'Mr. Anil Sharma', 'Physics', 'teach3')
(4, 'Ms. Kavita Iyer', 'Chemistry', 'teach4')
(5, 'Mr. Suresh Menon', 'Biology', 'teach5')
(6, 'Ms. Priya Kapoor', 'English', 'teach6')
(7, 'Mr. Vivek Das', 'History', 'teach7')
(8, 'Ms. Shalini Verma', 'Geography', 'teach8')
(9, 'Mr. Arvind Gupta', 'Economics', 'teach9')
(10, 'Ms. Ritu Sen', 'Political Science', 'teach10')
(11, 'Mr. Manoj Bhat', 'Physical Education', 'teach11')
(12, 'Ms. Neha Malhotra', 'Art & Culture', 'teach12')
(13, 'Mr. Sanjay Kulkarni', 'Hindi', 'teach13')
(14, 'Ms. Rekha Choudhary', 'Sanskrit', 'teach14')
(15, 'Mr. Ajay Prasad', 'Commerce', 'teach15')
(16, 'Ms. Bhavna Reddy', 'Business Studies', 'teach16')
(17, 'Mr. Deepak Mishra', 'Psychology', 'teach17')
(18, 'Ms. Anjali Sinha', 'Sociology', 'teach18')
(19, 'Mr. Harish Venkatesh', 'Music', 'teach19')
```

--- Admin Menu ---

1. Overall tally
 2. House Captain tallies (by house)
 3. Sports Captain tally
 4. School Prefect tally
 5. View Students
 6. View Teachers
 7. View Candidates
 8. Alter entry
 9. Delete entry
 10. Exit
- Choice: 7

Candidates Table:

(1,	'Patel Krishna',	'House Captain',	'Pioneers')
(2,	'Aditi Sen',	'House Captain',	'Pioneers')
(3,	'Nikita Das',	'House Captain',	'Explorers')
(4,	'Yash Arora',	'House Captain',	'Explorers')
(5,	'Rajat Kapoor',	'House Captain',	'Challengers')
(6,	'Sanya Kulkarni',	'House Captain',	'Challengers')
(7,	'Anushka Nair',	'House Captain',	'Voyagers')
(8,	'Pranav Joshi',	'House Captain',	'Voyagers')
(9,	'Raghav Sethi',	'Sports Captain',	'Pioneers')
(10,	'Ishita Rao',	'Sports Captain',	'Explorers')
(11,	'Tanvi Iyer',	'Sports Captain',	'Challengers')
(12,	'Neha Kapoor',	'Sports Captain',	'Voyagers')
(13,	'Rahul Rao',	'School Prefect',	'Pioneers')
(14,	'Meera Patel',	'School Prefect',	'Explorers')
(15,	'Ishita Sen',	'School Prefect',	'Challengers')
(16,	'Rahul Mehta',	'School Prefect',	'Voyagers')

--- Admin Menu ---

1. Overall tally
2. House Captain tallies (by house)
3. Sports Captain tally
4. School Prefect tally
5. View Students
6. View Teachers
7. View Candidates
8. Alter entry
9. Delete entry
10. Exit

Choice: 8

Table (students/teachers/candidates): candidates
candidate_id: 1

Field to update: name

New value: Krishna Prasad

☒ Updated.

--- Admin Menu ---

1. Overall tally
2. House Captain tallies (by house)
3. Sports Captain tally
4. School Prefect tally
5. View Students
6. View Teachers
7. View Candidates
8. Alter entry
9. Delete entry
10. Exit

Choice: 9

Table (students/teachers/candidates): students
student_id: 84

☒ Deleted.

--- Admin Menu ---

1. Overall tally
2. House Captain tallies (by house)
3. Sports Captain tally
4. School Prefect tally
5. View Students
6. View Teachers
7. View Candidates
8. Alter entry
9. Delete entry
10. Exit

Choice: 10



LEARNING OUTCOMES

By completing this project, one will:

1. Gain deeper understanding of Python programming and its application in database management.
2. Learn to design and implement a relational database using MySQL.
3. Develop skills in integrating Python with MySQL for real-world applications.
4. Enhance problem-solving abilities and technical project management skills.



BIBLIOGRAPHY

[https://cbseacademic.nic.in/web_material/doc/cs/2
Computer_Science_Python_ClassXII.pdf](https://cbseacademic.nic.in/web_material/doc/cs/2_Computer_Science_Python_ClassXII.pdf)

<https://cbseacademic.nic.in>